

Monitorización de eventos de seguridad

Ciberinteligencia y Herramientas de Análisis



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

School of Professional & Executive Development

René Serral <rene.serral@upc.edu>

Módulo 3: Sistemas de detección y monitorización

- **Módulo 3.1:** Introducción a los sistemas de detección y monitorización
- **Módulo 3.2:** Sistemas de detección y monitorización: Snort
- **Módulo 3.3:** Sistemas de detección y monitorización: OSSEC



Todos los créditos por estas slides van a:
Manel Rodero



Módulo 3.1: Introducción a los sistemas de detección y monitorización



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

School of Professional & Executive Development

Detección de intrusiones

- El principio de “defensa en profundidad” propone varias capas de mecanismos de defensa
 - Capa de **Prevención de ataques** – Firewall
 - Capa de **Detección de intrusiones** – IDS
- Entonces la detección de intrusiones debe asistir a la prevención de ataques cuando ésta no ha podido detectar el problema

Defensas en profundidad ■■■

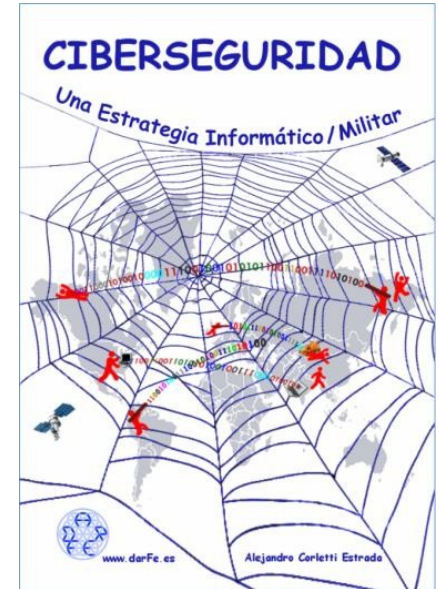
- Estrategia militar defensiva para asegurar una posición utilizando múltiples mecanismos de defensa
- Menos énfasis en una única protección perimetral
- Diversidad de barreras y diferentes tipos de fortificaciones
- Objetivo de ganar la batalla por desgaste
 - El atacante puede superar varios obstáculos, pero lo tiene difícil para sostener el ataque en el tiempo

Defensas en profundidad ■■■

Capa	Mecanismo
Datos	• Seguridad BD (almacenamiento en línea y copias de seguridad) • Seguridad del contenido (Information Rights Management)
Aplicación	• Federación (SSO, Confianza, ...) • Autenticación, Autorización, Auditoría (AAA) • Seguridad del código
Host	• Plataforma del SO, Gestión de las vulnerabilidades (patches), Escritorio (malware), etc.
Red Interna	• TLS (encriptación, identidad) • Cortafuegos, NAT, prevención de DoS, etc
Perímetro	
Física	• Vallas, candados, llaves, targetas, etc.
Políticas, procedimientos y conciencia	• Clasificación de datos, longitud de contraseñas, revisión de código, etc.

Defensas en profundidad ■■■

- "Ciberseguridad. Una estrategia informático/militar"
 - Noviembre 2017, [PDF](#), 245 pág.
 - Alejandro Corletti
 - Explica la ciberseguridad relacionando temas militares con su forma de implementarlos a través de diferentes protocolos de telecomunicaciones para lograr una adecuada estrategia de ciberdefensa



Ejemplos de intrusiones

- Acceso no autorizado a recursos
 - Descifrado/robo de contraseñas
 - Engaño (p.e. [DNS spoofing](#) = DNS cache poisoning)
 - Escucha del tráfico de red (sniffer)
- Modificación no autorizada de los recursos
 - Alteración o borrado de información
 - Cambios de configuración a sistemas y servicios de red
- Denegación de servicio (DoS)
 - Flooding: ping, spam
 - Compromiso del sistema: Buffer overflow
- Ataque a aplicaciones web
 - Defacement de servidor web (contenido inapropiado)



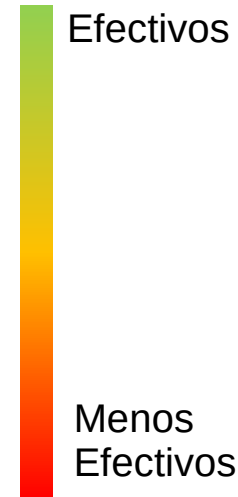
Intrusion Detection System (IDS)

- Una **intrusión** es cualquier intento o acceso no autorizado a un sistema así como el uso malicioso de los recursos de éste
- Los IDS son dispositivos hardware o aplicaciones software que **automatizan** el proceso de **monitorización** de los eventos que ocurren en la red y en los host, analizándolos para identificar signos de actividades maliciosas
 - Intentos por comprometer la **confidencialidad**, la **integridad** o la **disponibilidad**
 - Intentos por evitar los mecanismos de seguridad de un ordenador o de la red

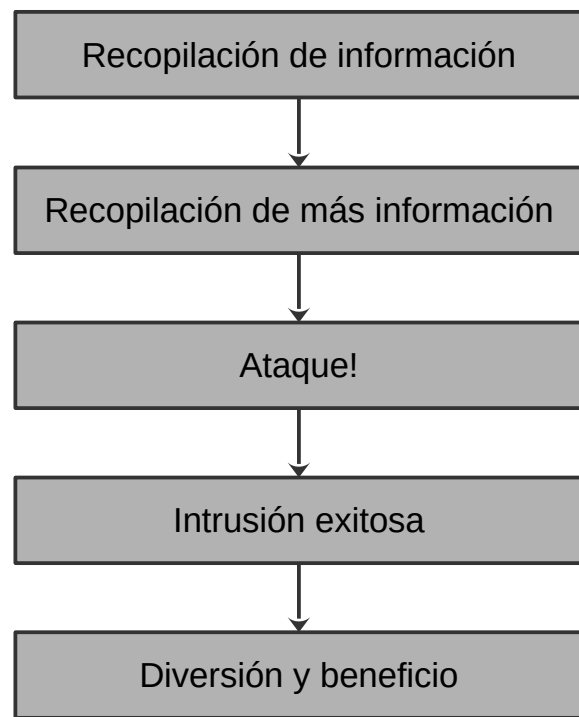


Efectividad de los IDS

- Depende del tipo de amenaza
 - Amenazas conocidas y ataques poco sofisticados
 - Grupos activistas
 - Estafas por correo electrónico a gran escala
 - Ataques dirigidos más sofisticados
 - Criminales
 - A/Entre estados
 - Nuevas vulnerabilidades
 - 0-day exploits



Escenario de intrusión típico



- Obtener el máximo de información posible
- Búsqueda whois y zonas DNS
- Navegación normal; obtención de información importante

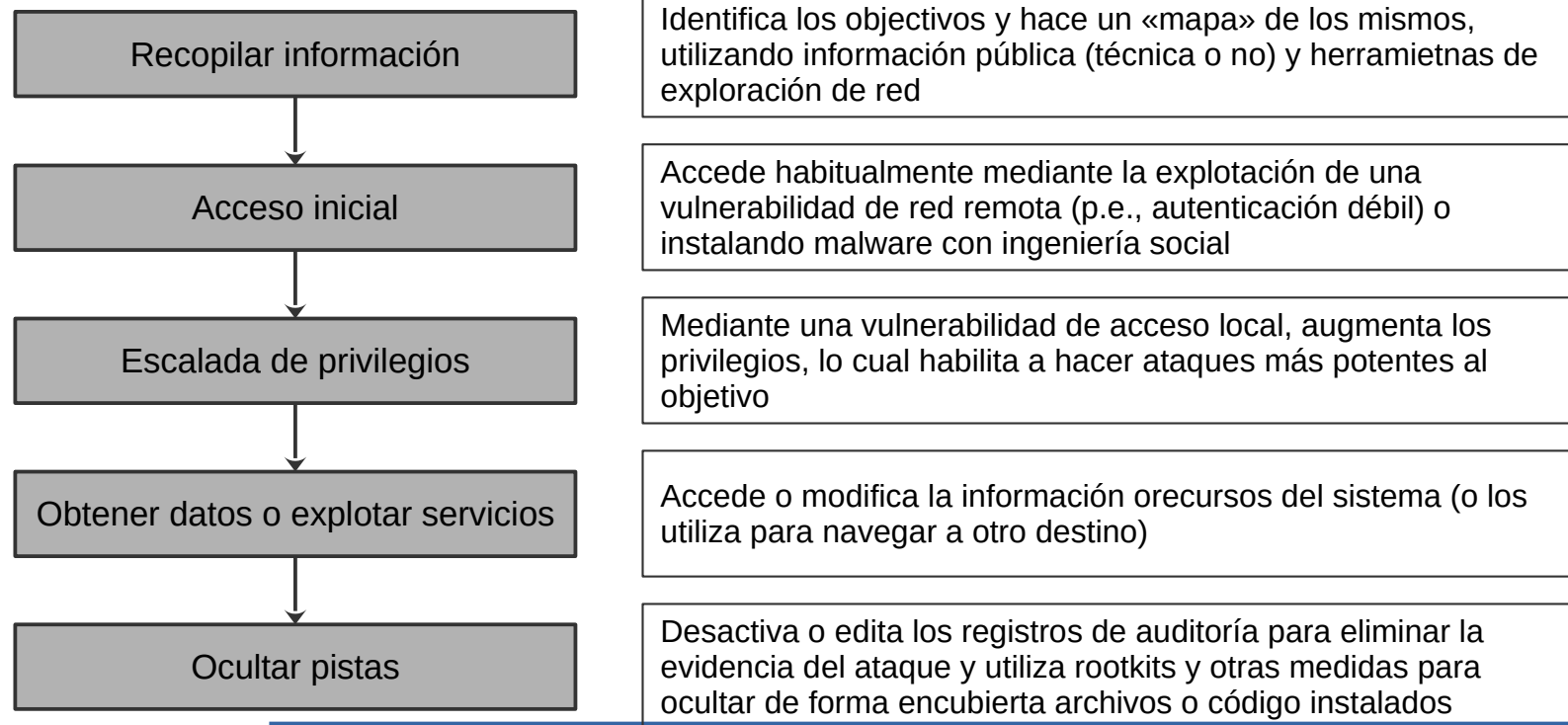
- Barridos de ping y escaneo de puertos
- Vulnerabilidades de los servicios expuestos
- Versiones de las aplicaciones ofreciendo el servicio

- Empezar a probar diferentes ataques
- Intentar encontrar servicios mal configurados
- Ataques pasivos y activos

- Instalar backdoors y borrar registros del paso
- Reemplazar servicios existentes por troyanos propios
- Crear nuevas cuentas de usuario

- Robar información confidencial
- Usar el host comprometido para lanzar otros ataques
- Cambiar la página web (defacing)

Comportamiento del intruso



Clasificación de los IDS

- Según el “**dónde**” se ejecutan (Despliegue)
 - Basados en host: HIDS (Host Intrusion Detection Systems)
 - Información “local” de log de auditoría del sistema
 - Basado en red: NIDS (Network Intrusion Detection Systems)
 - Información obtenida monitorizando el tráfico de la red a la que están conectados los host
- Según el “**cómo**” realizan la detección (Algoritmos)
 - Basados en firmas
 - Búsqueda de patrones dentro de la entrada
 - Basados en anomalías
 - Comportamiento no habitual o anómalo de un servicio
- Hay un tercer tipo emergente basado en el **aprendizaje**
 - Usando técnicas de machine learning para aprender de los datos

IDS basados en firmas

- Búsqueda de patrones o series de datos conocidas de antemano
- Búsqueda de strings o patrones de bits conocidos
 - Pueden contener expresiones regulares
- No mantienen estado, por lo que no disponen de lógicas complejas

Pros

- Muy pocos falsos positivos
 - Efectivos para amenazas conocidas
- Eficientes i rápidos
- Aceptación del mercado

Cons

- Muy poco efectivos para nuevas amenazas
 - Las amenazas puede ofuscarse con técnicas de evasión
- Incapacidad de detectar 0days



IDS basados en anomalías ■■■

- Muy efectivos para detectar amenazas desconocidas o 0day
 - Infección con un malware nuevo que para funcionar consume mucha CPU y genera una cantidad anómala de tráfico de red para generar SPAM
- Por contraparte puede generar muchos falsos positivos
- A la práctica crear un perfil de usuario es muy complicado
 - Muchas posibilidades, por aplicación, por sistema. ...



IDS basados en anomalías ■■■

- Fase de **entrenamiento**
 - Obtención del modelo de comportamiento del usuario
 - Recolección y procesamiento de indicadores durante el uso normal del sistema
 - En momentos diferentes
 - Proceso continuo de seguimiento y evolución del modelo
- Fase de **detección**
 - Comparación del estado del sistema con el modelo para detección. Puede ser:
 - Estadísticamente: series temporales de parámetros
 - Basándose con el conocimiento previo con reglas de clasificación según el entrenamiento previo
 - Con data mining usando técnicas “Machine Learning”

IDS basados en anomalías ■■■

Clasificación	Ventajas	Desventajas
Estadísticas	• Relativamente simple • Bajo coste computacional • No requiere información comportamiento esperado	• Dificultad en la selección de las métricas adecuadas: balance entre falsos positivos y falsos negativos • No todos los comportamientos pueden ser modelizados
Basada en conocimiento	• Robustez • Flexibilidad	• Dificultad y tiempo requeridos para desarrollar el conocimiento de alta calidad a partir de los datos • Requiere expertos humanos para ayudar con el proceso
Machine learning	• Flexibilidad • Adaptable • Interrelación entre los indicadores	• Dependencia de los supuestos sobre el comportamiento aceptado • Caro en recursos computacionales y tiempo para entrenar el sistema

IDS basados en red (NIDS)

- Actúan en la red, cercanos a los dispositivos que se quieren monitorizar
- Monitorizan la red, un segmento o un dispositivo
- La tarjeta de red necesita estar en modo promiscuo para poder capturar todo el tráfico
- Analiza los paquetes para detectar
 - Firmas de strings
 - Firmas de puertos
 - Firmas de condición de campos arbitrarios de la cabecera

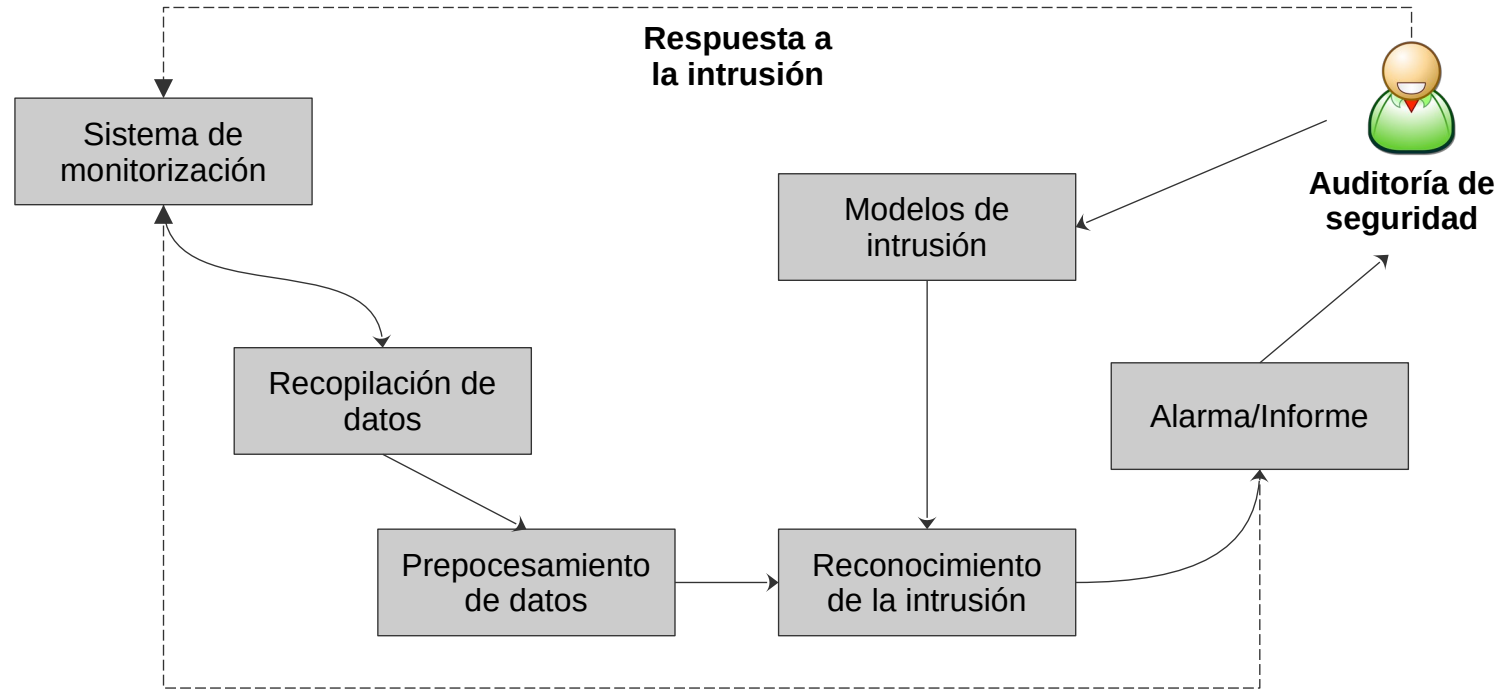
NIDS: Firmas

- Firmas de string: match de strings que pueden indicar ataque
 - Ej. "cat" "++" ">/ .rhosts"
 - Ej. "GET /etc/passwd"
- Firmas de puerto
 - Intentos de conexión a un conjunto de puertos
- Firmas de condición de cabecera
 - Combinaciones peligrosas o ilógicas

IDS basados en host (HIDS)

- Se ejecuta directamente en el host a analizar
- Utiliza los ficheros de log y el tráfico de E/S de la red de este host como fuente de datos
- Monitoriza
 - Paquetes de red entrantes
 - Actividades de inicio o final de sesión
 - Actividades del administrador
 - Actividad en el sistema de ficheros
 - Procesos en ejecución

Arquitectura de un IDS

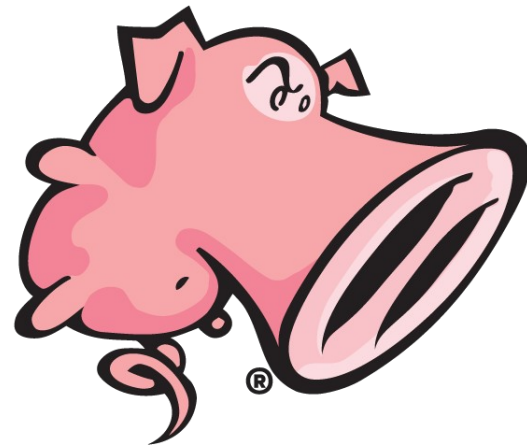


IDS Open Source

Network	Descripción
Snort	IDS/IPS disponible desde 1998 con amplio soporte por parte de la comunidad y probado a fondo. Aunque no tiene una consola gráfica, hay una gran cantidad de front-ends (p.ej. Snorby)
Suricata	Múltiples hilos de ejecución. Aceleración hardware usando GPU. Extracción de ficheros. Puede capturar mucho más que paquetes (p.ej. certificados TLS/SSL). Integración con logstash
Kismet	IDS especializado con el análisis de protocolos wireless 802.11

Host	Descripción
OSSEC	HIDS por excelencia. Arquitectura cliente/servidor. Agentes multiplataforma. Análisis de log, integridad de ficheros, monitorización de políticas, detección de rootkits, ...
Samhain	Arquitectura cliente/servidor. Agentes con múltiples métodos de salida (servidor central, Syslog, etc.). Análisis realizado en el cliente
Wazuh	Solución de código abierto y gratuita para la detección de amenazas, monitorización de la integridad, respuesta a incidentes y cumplimiento. Es un fork de OSSEC

Módulo 3.2: Sistemas de detección y monitorización: Snort



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

School of Professional & Executive Development

Snort: Características

- NIDS/NIPS* muy ligero y extensible
- Características
 - Analiza en tiempo real los paquetes que pasan por la red
 - No requiere cambios grandes en la configuración del sistema
 - Trabaja totalmente en espacio de usuario
 - Habilita el modo promíscuo para hacer la monitorización
 - Requiere que el tráfico de la red llegue al host – puede ser una copia
 - Requiere permisos de súperusuario para poderse ejecutar
 - Soporta inspección basada en firmas – content matching y content searching

* Network Intrusion Protection System (NIPS)



Snort

- Modos de funcionamiento
 - Packet sniffer: similar a Wireshark
 - Packet logger: guardar los resultados a ficheros
 - NIDS: detección y análisis del tráfico
- Requisitos
 - LibPCAP (o winPCAP) – Usado para la captura del tráfico
 - PCRE (Perl Compatible Regular Expressions)
 - libdnet: acceso a primitivas de red de bajo nivel
 - Barnyard2: formato unificado de salida en fichero binario
 - DAQ: API para la adquisición de datos

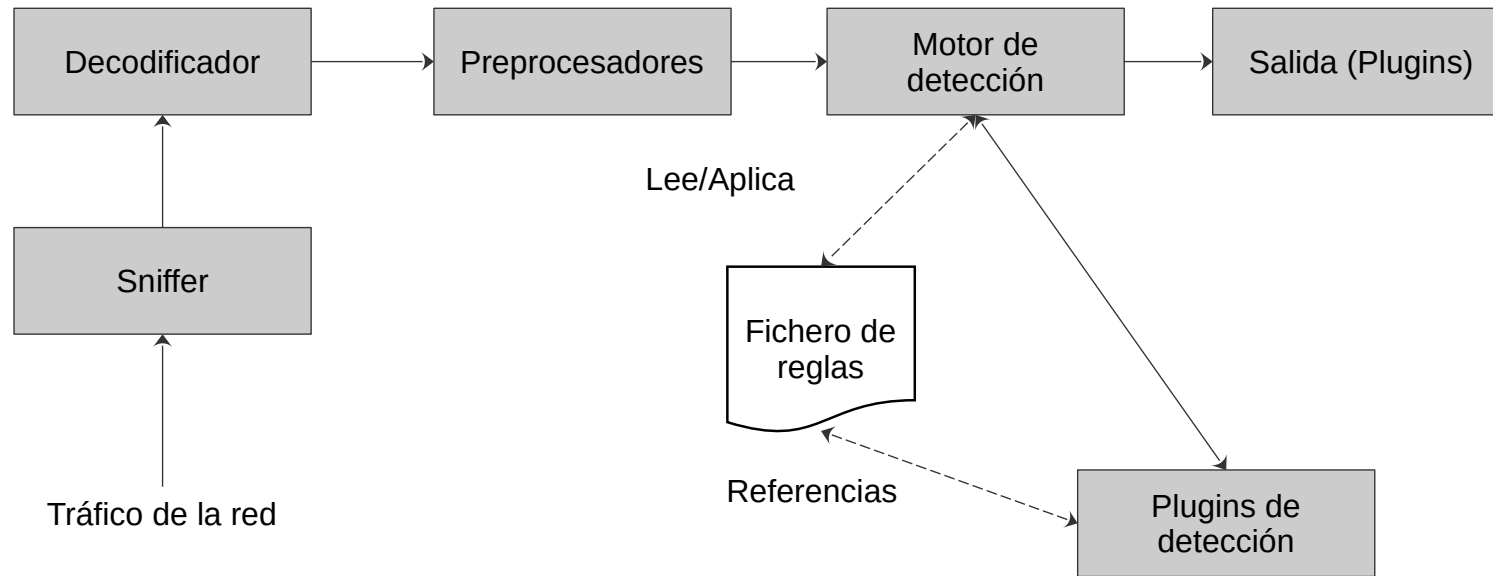
Snort: Reglas

- VRT (**Talos**) Certified Rules
 - Conjunto de reglas hechas por la comunidad útiles en varias situaciones
 - Subscriptores: las obtienen en tiempo real (499\$/año)
 - Usuarios registrados: 30 días más tarde
- **Pulledpork**
 - Se pueden descargar con este script
 - Requiere un **Oinkcode** de usuario registrado: snort.org



Snort: Arquitectura

- Compuesto por un core + plugins para extenderlo



Snort: Arquitectura

Componente	Descripción
Sniffer	Recoge el tráfico de red usando librerías externas (PCAP). Los paquetes sin procesar se envían al decodificador
Decodificador	Pipeline de decodificadores dependiendo de la capa: Datalink, Network, Transport. Almacena en una estructura de datos la información de cada capa
Preprocesadores	Analizan las estructuras para identificar qué clase de paquetes han llegado
Motor de detección	Es el componente principal del sistema. Analiza las reglas y comprueba si hay un match en la detección de firmas. Las firmas se construyen a partir de las reglas (cabecera y opción). Si hay un match el paquete se envía a la salida
Salida (Plugins)	Módulo que genera la salida hacia el exterior: Alertas, mensaje de log, entrada en la base de datos, ...

Snort: Arquitectura

Salida	Descripción
XML	Salida en formato XML. Permite anonimización de direcciones IP
CSV	Salida delimitada por ,
Alert_syslog	Envía la salida a una facility de syslog con una cierta prioridad
Database	Vuelca la salida a una base de datos como MySQL, Oracle, ...
Unified	Pensado para ser eficiente. Genera una salida en formato binario propietario con dos ficheros: el fichero de alertas (resumen con IP origen, destino puertos y la alerta) y el fichero de registro de paquetes completos que hacen match

Snort: detalle de modos de funcionamiento

- Sniffer: dump por pantalla
 - Cabeceras IP, TCP, UDP e ICMP
 - `snort -v -i eth0`
 - Datos de plicación (payload)
 - `snort -vd -i eth0`
- Packet Logger: grabarlos a disco
 - Fichero binario
 - `snort -vd -i eth0 -L <file> -b`
 - Fichero ASCII
 - `snort -vd -i eth0 -L <file> \`
`-k ascii`
- NIDS
 - Con fichero de configuración
 - No se registran todos los paquetes
 - Se aplican las reglas a cada paquete
 - Se deciden las acciones a ejecutar
 - Ejecución básica
 - `snort -d -L <file> -c \`
`/etc/snort.conf`
 - Comprobación de las reglas
 - `snort -d -L <file> -T -c \`
`/etc/snort.conf`

Snort: Reglas

- Muy flexibles y “fáciles” de modificar
 - Ejemplo de detección del troyano SubSeven

Cabecera de regla

Acción Protocolo IP Origen Puerto Origen IP Destino Puerto Destino

```
alert tcp $EXTERNAL_NET 27374 -> $HOME_NET any
(msg:"BACKDOOR subseven 22"; flags:A+; content:"|0d0a5b52504c5d3030320d0a|";
reference:arachnids,485; reference:url,www.hackfix.org/subseven/; sid:103;
classtype:misc-activity; rev:4;)
```

Opciones
De regla

Mensaje de alerta y información de qué
parte del paquete debe ser inspeccionado

Snort: Reglas

Elemento	Descripción
alert	Acción a realizar. Ver acciones
tcp	Protocolo, puede ser también udp, icmp, ip
\$EXTERNAL_NET	Dirección origen: variable
27374	Puerto origen. Podemos negarlo (!21) o un rango (1:1024)
->	Dirección del tráfico. No hace falta tocarlo (soporta <- y <>)
\$HOME_NET	Dirección destino: variable
any	Cualquier puerto destino

```
alert tcp $EXTERNAL_NET 27374 -> $HOME_NET any
(msg:"BACKDOOR subseven 22"; flags:A+; content:"|0d0a5b52504c5d3030320d0a|";
reference:arachnids,485; reference:url,www.hackfix.org/subseven/; sid:103;
classtype:misc-activity; rev:4;)
```



Snort: Opciones de regla

Elemento	Descripción
msg: "...";	Acción a realizar. Ver acciones
flags: A+;	Flags TCP, hay unos cuantos: SA, SA+, A → ACK, + → Any others
content: " x... ";	Datos binarios a comprobar en el paquete, sin el se comparan cadenas
reference: ...;	Información extra de la regla
sid: 103;	Identificador de la regla
classtype: ...;	Tipo de regla; hay muchos otros
rev: 4;	Versión de la regla

```
alert tcp $EXTERNAL_NET 27374 -> $HOME_NET any
(msg:"BACKDOOR subseven 22"; flags:A+; content:"|0d0a5b52504c5d3030320d0a|";
reference:arachnids,485; reference:url,www.hackfix.org/subseven/; sid:103;
classtype:misc-activity; rev:4;)
```



Snort: Acciones

Acción	Descripción
alert	Genera un alerta usando el método seleccionado y después registra el paquete
log	Registra el paquete
pass	Ignora el paquete
activate	Alerta y después activa una regla dinámica
dynamic	Regla que permanece inhabilitada hasta que es activada por otra regla del tipo «activate»; después actúa como regla de tipo «log»
drop	Bloquea y registra el paquete
reject	Bloquea el paquete, lo registra y después envía un TCP Reset si el protocolo es TCP o un ICMP Port Unreachable si es UDP
sdrop	Bloquea el paquete pero no lo registra

Snort: Referencias

- Permiten incluir información externa de identificación de vulnerabilidades

Acción	Descripción
bugtraq	http://www.securityfocus.com/bid/[ID]
cve	http://cve.mitre.org/cgi-bin/cvename.cgi?name=[CVE]
arachnids	Obsoleto
mcafee	Obsoleto
osvdb	http://osvdb.org/show/osvdb/[osvdb]
msb	http://technet.microsoft.com/security/bulletin/[msb]
url	http://[url]

Snort: detección de contenido – content

- Busca un contenido concreto dentro del **payload** del paquete y genera una respuesta basada en esto

```
alert tcp 192.168.1.0/24 any -> any any (content:"HTTP";  
offset:4; msg:"HTTP matched";)
```

- Modificadores
 - **offset**: posición del paquete donde empezar a buscar el patrón
 - **depth**: posición del paquete asta la cual buscar el patrón
 - **distance**: posición del paquete donde empezar a buscar el patron pero relativo a la búsqueda anterior
 - **within**: asegura que como máximo hay N bytes entre coincidencias de contenido

Snort: detección de contenido

- Usando content debemos considerar que
 - La búsqueda de contenido es un proceso costoso computacionalmente. No conviene poner demasiadas reglas
 - Si el contenido es ASCII, hay que escapar caracteres especiales: \", \, , \ |
 - Se puede usar varias veces dentro de una regla para encontrar múltiples firmas
 - La búsqueda es case sensitive



Ejemplos reales de uso de Snort

- El 2 Feb 2017 se publica un **0day en SMB**
- Al día siguiente **Florian Roth**, creador de **Sigma**, publica una regla para detectarlo
- Un mes después **14 Mar 2017** Microsoft soluciona la vulnerabilidad



Florian Roth

@cyb3rops

Ah, @johullrich provided a Snort rule for the SMB 0day yesterday
isc.sans.edu/diary/Windows%...

Traducir del inglés

This is where the message should end. But apparently, since the total message size according to the NetBIOS header is larger, Windows keeps on decoding in the crafted header (all "C"s in the exploit) which then triggers the buffer overflow.

Based on this understanding of the exploit (please let me know if I didn't get it right or missed something), I wrote a simple snort signature that looks for Tree Connect messages that exceed 1000 bytes in size. Use this at your own risk. It is in "works for me" state:

```
alert tcp $EXTERNAL_NET 445 -> $HOME_NET any (sid: 10001515; msg: "SMB Excessive Large Tree Connect Response"; byte_test: 3,>,1000,1; content: "|fe 53 4d 42 40 00|"; offset: 4; depth: 6; content: "|03 00|"; offset: 16; depth:2 ;)
```

RETWEETS

107

ME GUSTA

109



19:08 - 3 feb. 2017



1



107



109



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

School of Professional & Executive Development

Usando Snort



- Instalación
- Modo «sniffer»
- Modo «logger»
- Modo NIDS
- Detección ICMP, HTTP
- Detección de contenido

Módulo 3.3: Sistemas de detección y monitorización: OSSEC



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

School of Professional & Executive Development

OSSEC: Características

- Open source
- Análisis de logs
- Comprobación de la integridad de ficheros
- Comprobación de la integridad del registro
- Monitorización/aplicación de políticas
- Detección de anomalías en el host
- Detección de rootkit
- Alertas en tiempo real
- Respuestas activas



OSSEC. Sistemas soportados

- Sistemas operativos
 - Linux, Windows, BSC, MacOSX, Solaris, ...
- Dispositivos soportados via syslog
 - Cisco PIX, Cisco IOS, Juniper Netscreen, SonicWall Firewall, Checkpoint Firewall, Sourcefire, Snort, McAfee Viruscan
- Agentless (análisis de logs e integridad de ficheros)
 - Cisco PIX, Cisco IOS, Checkpoint Firewall, todos los SO anteriores
- Monitorización de BD
 - MySQL, PostgreSQL, Oracle, MSSQL

OSSEC: Formatos soportados

Familia	Tipos
Unix-only	Unix PAM, sshd, samba, su, sudo, cron/crontab, dpkg logs, etc...
FTP servers	proftpd, wu-ftpd, Microsoft FTP server, etc
Mail servers	imapd and pop3d, postfix, sendmail, Microsoft Exchange, etc.
Web servers	Apache web server (access log and error log), IIS 5/6 web server (NSCA and W3C extended)
Web applications	Horde imp, Modsecurity, etc
Firewalls	iptables firewall, Solaris ipfilter firewall, Windows firewall, etc.
Databases	MySQL, PostgreSQL
NIDS	Cisco IOS IDS, Snort IDS, etc
Security Tools	Symantec Antivirus, Nmap, McAfee VirusScan Enterprise, etc
Others	named (bind), Squid proxy, VMWare ESX, etc
Windows	Windows event logs, RAS, etc

OSSEC: Componentes ■■

- Gestor central (Manager)
 - Base de datos de integridad de ficheros
 - Logs, eventos y entradas del sistema de auditoría
 - Reglas, decodificadores y opciones de configuración
- Agentes
 - Pequeños programas instalados en el sistema a monitorizar
 - Recogen información en tiempo real y la envían al gestor para ser analizada y correlacionada
 - Poco uso de memoria y CPU

OSSEC: Componentes ■■

- Agentless
 - En sistemas donde no se puede instalar ninguna aplicación externa
 - Firewalls, routers o sistemas sin posibilidad de instalar agentes
- VMWare
 - Instalación en el host VMWare ESXi
 - Alertas de VM añadida, eliminada, iniciada...
- Firewalls, Switches y Routers
 - Recibe mensajes de syslog

OSSEC: Instalación

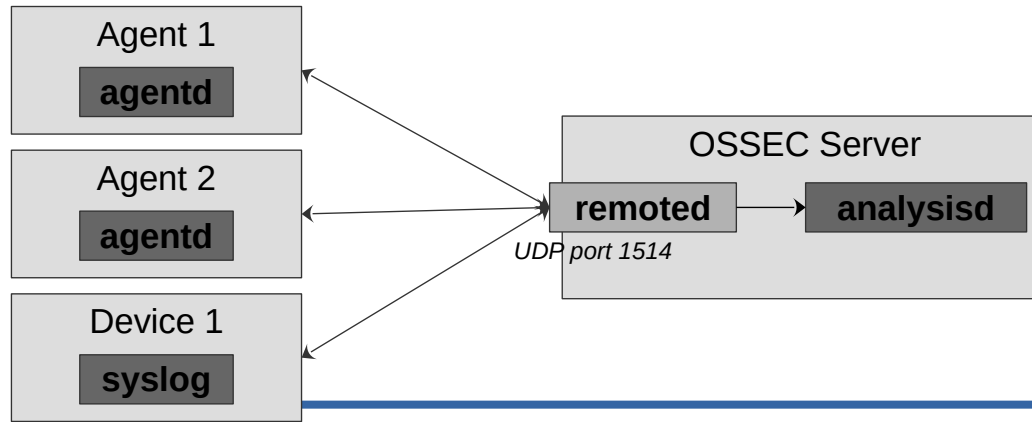
- Sencilla en cualquiera de sus variantes
 - Local → único sistema a monitorizar
 - Cliente/Servidor → análisis centralizado
- Seleccionar el tipo de instalación y responder algunas preguntas
- Script de instalación para el ajuste de permisos y la creación de los usuarios

OSSEC: Procesos

Proceso	Descripción
syscheckd	Se encarga de ejecutar los análisis de integridad
logcollector	Recoge los archivos de log sin analizarlos: syslog, ficheros planos, Windows evet log, IIS, etc (user root)
agentd	Solo en agentes: envía los registros al manager remoto (user ossec)
execd	Ejecuta las respuestas activas, p.e., bloqueo de direcciones IP
remoted	Recibe los logs de los agentes remotos (user ossecr)
analysisd	Proceso principal. A cargo de realizar todo el análisis (user ossec)
maild	Envía alertas por correo electrónico (user ossecm)
monitord	Monitoriza el estado de los agentes, comprime y forma los ficheros de log

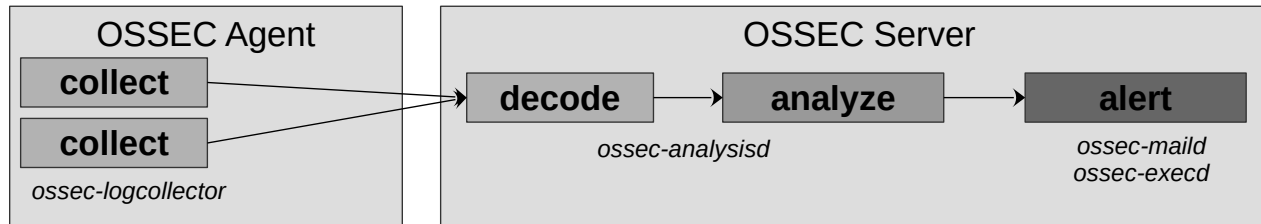
OSSEC: Comunicación por red

- La comunicación entre agentes y servidor
 - Se comprime usando zlib y se encripta usando claves generadas previamente de tipo blowfish
 - Utiliza el puerto UDP 1514 por defecto
 - Es multiplataforma



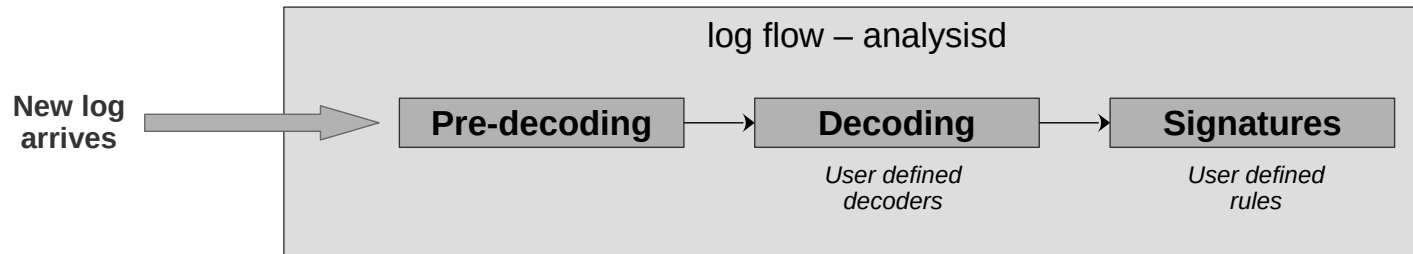
OSSEC: Flujo de logs

- Recogida de los log: `ossec-logcollector`
- Análisis y decodificación: `ossec-analysisd`
- Alertas realizadas mediante: `ossec-maild`
- Respuestas activas mediante: `ossec-execd`

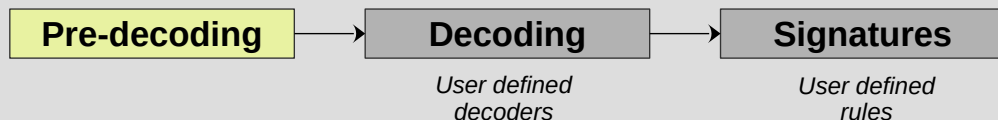


OSSEC: Flujo de logs interno

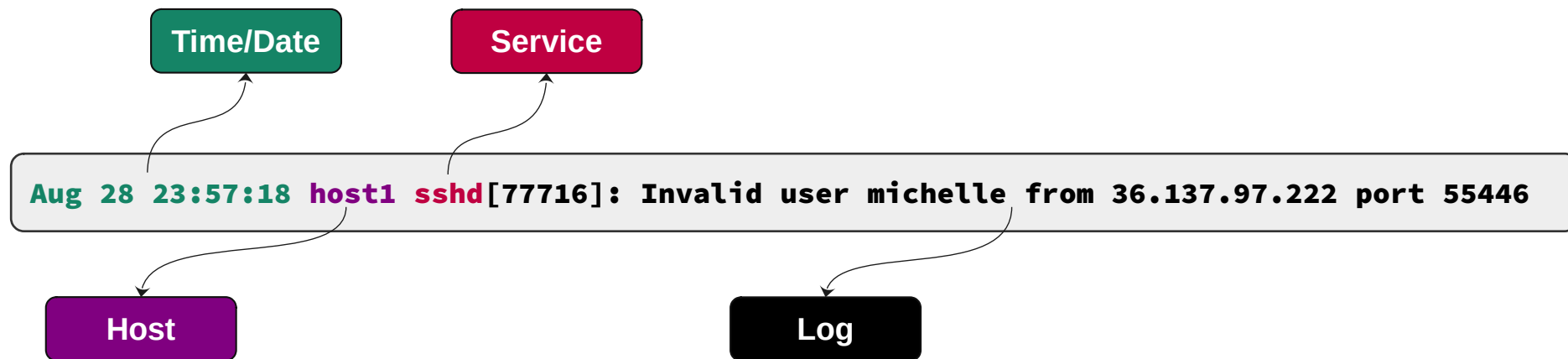
- **Decodificación previa:** extrayendo campos conocidos: fecha, dirección IP, ...)
- **Decodificación:** usando expresiones definidas por el usuario en formato XML
- **Firmas:** usando reglas definidas por el usuario



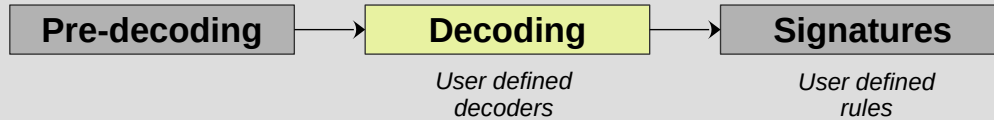
log flow – analysisd



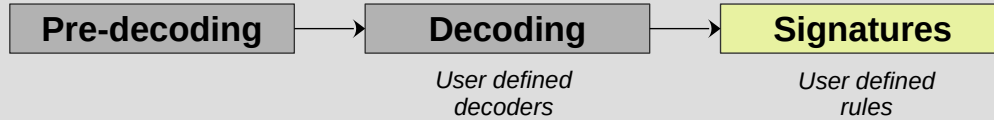
- Extracción de información genérica de las cabeceras del syslog
 - Nombre del host, fecha, ...
- Imprescindible tener los log bien formateados



log flow – analysisd



- Tiene por objetivo identificar la información clave del log
- Hay muchísimos decodificadores en decoders.xml
- El usuario puede definir de propios
- Normalmente se adquiere la IP, el usuario, la aplicación...
- Por ejemplo:
 - **Time/Date:** Aug 28 23:57:18
 - **Hostname:** host1
 - **Program name:** sshd
 - **Log:** Invalid user michelle from 36.137.97.222 port 55446
 - **SRC IP:** 36.137.97.222
 - **User:** michelle



- Tiene por objetivo comprobar las firmas usando las reglas definidas por el usuario en ficheros XML
 - Por defecto existen más de 400 reglas
 - Tienen niveles del 1 al 15
 - Por los decodificadores son independiente del formato inicial
 - Permiten hacer matching usando la información decodificada
- Tipos de reglas
 - **Atómicas:** basadas en un evento simple
 - **Compuestas:** basadas en patrones de múltiples logs

OSSEC: Reglas

- Requisitos
 - Identificador de regla: entero
 - Nivel de regla: 0 – ignore, 15 – ataque con éxito
 - El nivel 0 no genera alerta pero puede dar información extra
 - Un patrón: cualquier expresión regular o campos a verificar:
 - srcip, id, user, ...
- Ejemplo
 - Si el log viene de la aplicación ssh entonces aplica la regla 111 de nivel 0

```
<rule id="111" level="0">  
  <decoded_as>sshd</decoded_as>  
  <description>Mensajes SSHD</description>  
</rule>
```

OSSEC Reglas en entornos reales ■■■■

- Control de autenticación: alarma a autenticación correcta fuera de horario laboral
 - Clasificación del evento como `authentication_success`
 - Se usa `if_group` si hace matching con el grupo indicado
- Uso del fichero `local_rules.xml`
- Nivel 10: errores generados por los usuarios

```
<rule id="100101" level="10">  
  <if_group>authentication_success</if_group>  
  <time>8:00pm - 08:00am</time>  
  <description>Login fuera de horario</description>  
</rule>
```


OSSEC Reglas en entornos reales ■■■■

- Control de autenticación II: alarma a la primera autenticación correcta fuera de horario laboral
 - Existen reglas FTS (First time seen)
 - Esta primer vez la alerta se considera más severa
- Uso del fichero `local_rules.xml`
- Nivel 13: errores inusuales (pero importantes)

```
<rule id="100102" level="13">  
  <if_sid>111, 100101</if_sid>  
  <time>8:00pm - 08:00am</time>  
  <description>Primer login fuera de horario</description>  
</rule>
```

OSSEC Reglas en entornos reales ■■■

- Reglas oficiales de firewalls

```
<group name="firewall,">
  <rule id="4100" level="0">
    <category>firewall</category>
    <description>Firewall rules grouped.</description>
  </rule>
  <rule id="4101" level="5">
    <if_sid>4100</if_sid>
    <action>DROP</action>
    <options>no_log</options>
    <description>Firewall drop event.</description>
    <group>firewall_drop,</group>
  </rule>
  <rule id="4151" level="10" frequency="16" timeframe="45" ignore="240">
    <if_matched_sid>4101</if_matched_sid>
    <same_source_ip />
    <description>Multiple Firewall drop events from same source.</description>
    <group>multiple_drops,</group>
  </rule>
</group>
```

OSSEC Reglas en entornos reales ■■■■

- Detección de fuerza bruta en ssh

```
-A INPUT -p tcp --dport 22 -j DROP --log-prefix "SSH: "
```

```
Oct 20 03:46:02 ubuntu16vm kernel: SSH: IN=eth0 OUT=  
MAC=xx:xx:xx:xx:xx:xx:xx:xx:xx:xx:xx:xx:xx:xx:xx:xx SRC=x.x.x.x  
DST=y.y.y.y LEN=52 TOS=0x00 PREC=0x00 TTL=52 ID=763 DF PROTO=TCP  
SPT=20229 DPT=22 WINDOW=15588 RES=0x00 ACK URG=0
```

- #ossec-test
 - Fase 1: decodificación previa
 - Full event, hostname, program name, log
 - Fase 2: decodificación
 - Decoder (iptables), action (SSH:), srcip, dstip, proto, srcport, dstport
 - Fase 3: filtro
 - Rule id (4100), Level (0), Description (Firewall rules grouped.)



OSSEC Reglas en entornos reales ■■■■

- Creación dinámica de nuevas reglas
 - Regla 4102: Obtención de la IP
 - Regla 4152: Si esta IP genera 20 incidencias en 1 día entonces se genera una alerta de nivel 10

```
<rule id="4102" level="5">
  <if_sid>4100</if_sid>
  <action>SSH:</action>
  <description>Conexiones SSH</description>
</rule>
<rule id="4152" level="10" frequency="20" timeframe="86400">
  <if_matched_sid>4102</if_matched_sid>
  <same_source_ip />
  <description>Fuerza bruta SSH</description>
</rule>
```

OSSEC: Detección de rootkits

- Mediante firmas (p.ej. [Knark](#)) o basándose en anomalías
- Los ficheros en /dev que no son dispositivos
- Directorios ocultos
- Ficheros [SUID](#)
- Ficheros que pertenecen a root escribibles por todo el mundo
- Procesos en ejecución ocultos al comando ps
- Puertos abiertos ocultos al comando netstat
- Interfaces en modo promiscuo
- Ejecutables en /tmp



OSSEC: Comprobación de integridad

- Cada cierto tiempo o en tiempo real
- Propiedades de ficheros y directorios
- Permisos
- Tamaño
- Propietario
- Usando hashes
 - sha256sum
 - sha512sum

```
<rule id="100345" level="12">  
  <if_matched_group>sys_check</if_matched_group>  
  <match>/var/www/htdocs</time>  
  <description>Cambios en /var/www/htdocs!</description>  
</rule>
```

OSSEC: Monitorización de políticas

- Identificar situaciones que pueden conducir a un incumplimiento de políticas de empresa
- Realizar un benchmark del sistema usando el estándar CIS o uno propio
- Criterios de ejemplo
 - Comprobar la existencia o no existencia de ficheros, configuración de registro o procesos (win_audit_rcl.txt, win_applications_rcl.txt)
 - Existencia de antivirus pero que esté parado
 - Firewall deshabilitado
 - ...

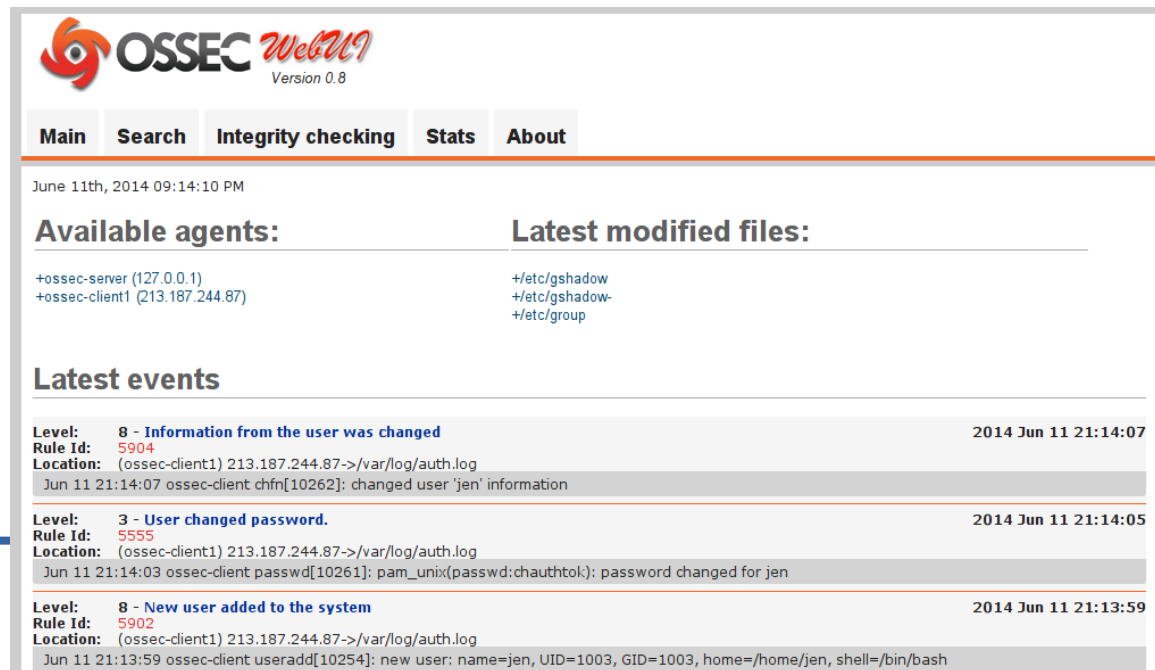
Jugando con los sistemas



- Instalación de wazuh (fork de ossec)
- Añadir agente Linux
- Añadir agente Windows
- Generar algunos registros

OSSEC: WebUI

- Interfaz gráfica muy sencilla incluida en OSSEC
- <http://www.ossec.net/wiki/index.php/OSSECWUI:Install>



The screenshot displays the OSSEC WebUI interface, Version 0.8. The top navigation bar includes links for Main, Search, Integrity checking, Stats, and About. The main content area shows the date and time as June 11th, 2014 09:14:10 PM.

Available agents:

- +ossec-server (127.0.0.1)
- +ossec-client1 (213.187.244.87)

Latest modified files:

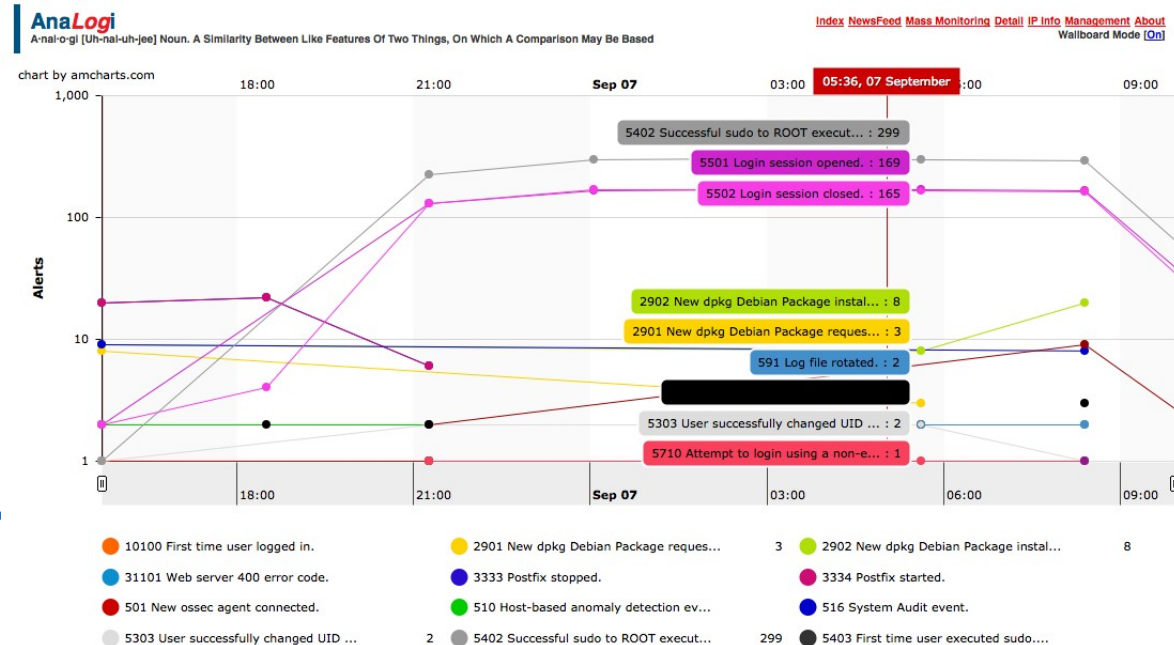
- +etc/gshadow
- +etc/gshadow-
- +etc/group

Latest events

Level: 8 - Information from the user was changed	2014 Jun 11 21:14:07
Rule Id: 5904	
Location: (ossec-client1) 213.187.244.87->/var/log/auth.log	
Jun 11 21:14:07 ossec-client chfn[10262]: changed user 'jen' information	
Level: 3 - User changed password.	2014 Jun 11 21:14:05
Rule Id: 5555	
Location: (ossec-client1) 213.187.244.87->/var/log/auth.log	
Jun 11 21:14:03 ossec-client passwd[10261]: pam_unix(passwd:chauthtok): password changed for jen	
Level: 8 - New user added to the system	2014 Jun 11 21:13:59
Rule Id: 5902	
Location: (ossec-client1) 213.187.244.87->/var/log/auth.log	
Jun 11 21:13:59 ossec-client useradd[10254]: new user: name=jen, UID=1003, GID=1003, home=/home/jen, shell=/bin/bash	

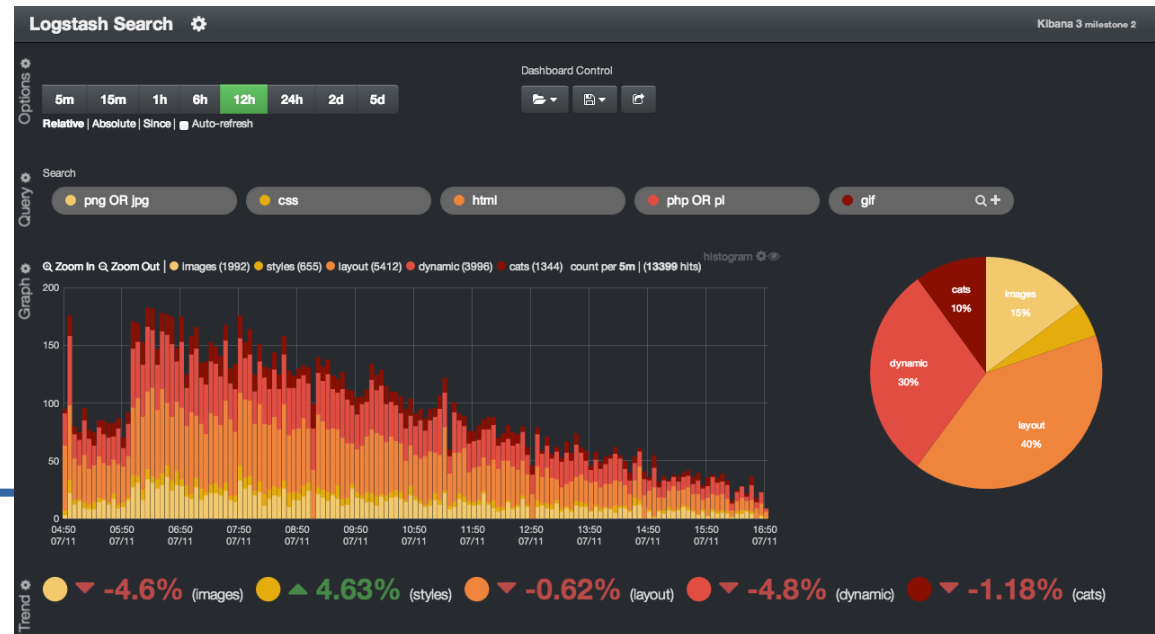
Analogi Web Dashboard

- Información más visual que la interfaz WUI
 - Requiere configurar OSSEC para usar MySQL como BD
 - <https://github.com/ECSC/analogi>



Kibana

- Almacenar, buscar y visualizar eventos de seguridad
 - Logstash (parser) + Elasticsearch (index) + Kibana (GUI)
 - <http://vichargrave.com/ossec-log-management-with-elasticsearch/>



Wazuh

- Visibilidad de seguridad basada en host, usa agentes ligeros multiplataforma
 - Fork de OSSEC
 - <https://wazuh.com/migrating-from-ossec/>

